

Q9: Deployment & Configuration Setup

Question: Please explain the deployment and configuration setup. How does the application manage environment variables (e.g., database credentials, secret keys)? Identify the server configuration (e.g., Uvicorn/Gunicorn settings) and summarize any Dockerfiles, containerization strategies, or CI/CD pipelines present in the repository.

Part 1: Environment Variable Management

Configuration Layer: `pydantic-settings`

All runtime configuration is centralised in a single `Settings` class defined in `[app/core/config.py](app/core/config.py)`:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    DATABASE_URL: str = Field()  # required — no default
    SECRET_KEY: str = Field()    # required — no default
    HARDWARE_API_KEY: str = Field()  # required — no default

    REDIS_URL: str = "redis://localhost:6379"
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 60
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7
    # ... 15+ additional typed fields ...

    model_config = SettingsConfigDict(
        env_file=".env",
        env_ignore_empty=True,
        extra="ignore"
    )

settings = Settings()
```

`Settings()` is instantiated once at import time and the `settings` singleton is imported wherever configuration values are needed. This means:

- Startup fails immediately and loudly if a **required field** (one without a default) is missing from the environment — no silent fallbacks.
- All values are **type-validated** by Pydantic on load — a non-integer `ACCESS_TOKEN_EXPIRE_MINUTES` raises a `ValidationError` before the server accepts any requests.
- `extra="ignore"` means unknown variables in `.env` are silently discarded, preventing import errors when the environment has extra OS-level variables.

Environment Variable Categories

Category	Variables	Required / Optional
Database	DATABASE_URL	Required
Cache / Messaging	REDIS_URL	Optional (default: redis://localhost:6379)
JWT	SECRET_KEY , ALGORITHM , ACCESS_TOKEN_EXPIRE_MINUTES , REFRESH_TOKEN_EXPIRE_DAYS	SECRET_KEY required; rest defaulted
CORS	BACKEND_CORS_ORIGINS	Optional (default: localhost:5173 + 3000)
Hardware Auth	HARDWARE_API_KEY	Required
Business Logic	BREAK_TIME_PER_SHIFT , MIN_TRIPS_BEFORE_BREAK , MIN_BREAK_DURATION , MAX_BREAK_DURATION , TURNAROUND_BUFFER , SHIFT_END_PROTECTION , CROWDING_THRESHOLD , EXTRA_DISPATCH_PREP_TIME , D3_SHIFT_OFFSET_HOURS , MORNING_SHIFT_START/END , EVENING_SHIFT_START/END	All optional with sane defaults
Scheduler	DAILY_SCHEDULE_GENERATION_HOUR /MINUTE , ASSIGNMENT_MAX_RETRIES , ASSIGNMENT_RETRY_DELAY	All optional with sane defaults

Source Resolution Order

pydantic-settings resolves values in this priority order (highest wins):

1. **OS environment variables** (set by the shell, Docker `-e` , systemd `Environment=` , etc.)
2. **.env file** in the working directory (loaded by `SettingsConfigDict(env_file=".env")`)
3. **Field default** defined in the `Settings` class

This means a CI/CD pipeline can override any `.env` value by setting an OS-level environment variable without touching the file.

The `.env` File

A committed `[.env.example](.env.example)` documents every expected variable with placeholder values and generation instructions:

```
# Generate with: python3 -c "import secrets; print(secrets.token_hex(32))"
SECRET_KEY="your_64_char_hex_secret_here"

# Generate with: python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

```
HARDWARE_API_KEY=your_cryptographic_api_key_here
```

The actual `.env` file is **not excluded from the repository** in this development setup — it contains live Supabase credentials and the JWT secret key. In a production deployment these would be injected via the deployment platform's secrets manager (Docker secrets, environment injection, etc.) rather than committed to source control.

Actual `.env` database URL (production-pointing):

```
DATABASE_URL="postgresql+asyncpg://postgres.kgqfhowapmngxrskwlfm:<password>@aws-1-eu-central-1.pooler.supabase
```

This is a **Supabase connection pooler** URL (PgBouncer on port 5432, EU Central 1 region) — the application connects to a managed cloud Postgres instance, not a local one.

Part 2: Server Configuration (Uvicorn)

What Is Used

The application uses **Uvicorn** as the sole ASGI server. There is no Gunicorn process manager, no `gunicorn.conf.py`, and no Procfile. The dependency is declared as:

```
uvicorn[standard]>=0.34.0,<1.0.0
```

The `[standard]` extra pulls in:

- `uvloop` — high-performance asyncio event loop (Linux/macOS)
- `httptools` — faster HTTP/1.1 parser
- `websockets` — WebSocket protocol support (required for the real-time notification system)

How the Server Is Started

There is no dedicated startup script in the repository. The standard invocation is:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

`--reload` is development-only. In production the `--reload` flag is dropped and worker count / log level would be passed as CLI arguments or via a process supervisor.

Why Not Gunicorn?

A `uvicorn[standard]` install with multiple workers can be achieved with:

```
uvicorn app.main:app --workers 4
```

Gunicorn is the traditional choice for multi-worker deployments (it manages Uvicorn workers via `uvicorn.workers.UvicornWorker`), but this codebase does not include that configuration. The single-process Uvicorn pattern is appropriate for a development/staging environment backed by a cloud-managed database.

Uvicorn's Role in Logging

As described in Q8, Uvicorn owns all logging configuration for the application. It provides:

Logger	Output
<code>uvicorn.access</code>	One line per HTTP request on stdout
<code>uvicorn.error</code>	All <code>WARNING</code> and above from application modules on stderr

No `logging.basicConfig()` or file handlers are configured in application code — everything propagates to Uvicorn's root handler.

Part 3: Database Migrations (Alembic)

Database schema is managed by **Alembic**, not by SQLAlchemy's `create_all()`. Tables are never auto-created at server startup.

Migration Configuration

alembic.ini configures:

- `script_location = alembic` — migration scripts live in `alembic/versions/`
- SQLAlchemy logging silenced at `WARN` level during migration runs
- Alembic's own log set to `INFO` (migration steps are visible)

alembic/env.py bridges Alembic to the application's async engine:

```
from app.core.config import settings          # reads DATABASE_URL from .env
from app.core.database import Base
from app.models import models                # ensures all ORM models are registered

target_metadata = Base.metadata              # drives autogenerate diff

def run_migrations_online():
    engine = create_async_engine(
        settings.DATABASE_URL,
        poolclass=pool.NullPool,              # no pool – migrations are one-shot
        future=True,
    )
    async def run_async_migrations():
        async with engine.connect() as conn:
            await conn.run_sync(do_run_migrations)
    asyncio.run(run_async_migrations())
```

Key design points:

- Uses `NullPool` during migrations — each migration command creates a fresh connection and closes it immediately, avoiding pool state issues.
- Reads `DATABASE_URL` from the same `settings` object the application uses — no separate connection string to keep in sync.
- `compare_type=True` in both online and offline modes — Alembic detects column type changes, not just additions/removals.

Migration Version History

File	Description
e4c72e85d44d_initial_schema.py	Initial schema
0001_initial.py	Foundation tables (Users, Drivers, Vehicles, Routes, Trips)
0002_add_fatigue_score.py	fatigue_score column on Drivers
6fe816cdcd0c_add_sprint_3_iot_and_camera_models.py	IoT sensor and CameraReading tables
0003_add_gate_logs_and_camera_readings.py	GateLogs and CameraReading refinements
0004_sprint4_p2_and_p3_model_changes.py	Sprint 4 model changes (P2/P3 features)

Migrations are run manually:

```
alembic upgrade head      # apply all pending migrations
alembic downgrade -1      # roll back one revision
alembic revision --autogenerate -m "description" # generate new migration from model diff
```

Part 4: Dependency Management

All Python dependencies are pinned with `>=min,<major_bump` version constraints in `requirements.txt`:

```
fastapi>=0.115.0,<1.0.0
uvicorn[standard]>=0.34.0,<1.0.0
sqlalchemy[asyncio]>=2.0.36,<3.0.0
asyncpg>=0.30.0,<1.0.0
alembic>=1.12.0,<2.0.0
pydantic>=2.10.0,<3.0.0
pydantic-settings>=2.7.0,<3.0.0
python-jose[cryptography]>=3.3.0,<4.0.0
bcrypt>=4.2.0,<5.0.0
apscheduler>=3.10.0,<4.0.0
slowapi>=0.1.9,<1.0.0
redis>=5.0.0,<6.0.0
bleach>=6.1.0,<7.0.0
```

Notable version decisions:

- `passlib` is **absent** — the comment in `requirements.txt` explains: "passlib broken on Python 3.13". `bcrypt` is used directly instead.
- `sqlalchemy[asyncio]` pulls in `greenlet`, needed for SQLAlchemy's async bridge.
- `python-multipart` is included specifically for `OAuth2PasswordRequestForm` (FastAPI dependency).

Part 5: Containerization & CI/CD

No Docker or CI/CD Present

The repository contains **no Dockerfile**, **no** `docker-compose.yml` , **no** `.github/workflows/` , and **no CI/CD pipeline configuration** of any kind. The deployment model is:

- **Backend:** Uvicorn process run directly on the host or development machine, pointed at a remote Supabase Postgres instance and a local/remote Redis instance.
- **Database:** Fully managed by [Supabase](https://supabase.com) (hosted Postgres on AWS EU Central 1). No local Postgres container is needed — the `DATABASE_URL` in `.env` points directly to Supabase's connection pooler.
- **Redis:** The `REDIS_URL` defaults to `redis://localhost:6379` — a locally-running Redis instance is expected. Redis is used for idempotency key storage and WebSocket pub/sub only; the application degrades gracefully if Redis is unavailable at startup (both `IdempotencyMiddleware` and the socket manager handle connection failures non-fatally).
- **Frontend:** Vite dev server (`npm run dev`) run separately on port 5173/5174.

Infrastructure Topology (Development)

Developer Machine

■■■

uvicorn app.main:app --port 8000

← Backend API

■

■■■ → Supabase (AWS EU-Central-1)

← Postgres (asyncpg over TLS)

■

■■■ → localhost:6379

← Redis (idempotency + pub/sub)

■■■

npm run dev (Vite) --port 5173

← Frontend (React/TypeScript)

■■■

→ http://localhost:8000

← API calls (CORS allowed)

Production Path (Not Implemented)

For a production deployment, the standard path for this stack would be:

1. **Containerize** — create a `Dockerfile` using `python:3.13-slim` , install `requirements.txt` , set `CMD` `["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]`
2. **Secrets** — inject `DATABASE_URL` , `SECRET_KEY` , `HARDWARE_API_KEY` via the platform's secrets manager (not `.env` files)
3. **Multi-worker** — use `uvicorn --workers N` or `Gunicorn + UvicornWorker` for horizontal scaling
4. **Redis** — provision a managed Redis instance (ElastiCache, Upstash, Redis Cloud) and update `REDIS_URL`
5. **CI/CD** — add GitHub Actions or similar to run `alembic upgrade head` on deploy and run test suites

None of these steps are currently implemented in the repository.

Summary

Aspect	Implementation
Config management	<code>pydantic-settings BaseSettings</code> , <code>.env</code> file + OS env override

Aspect	Implementation
Required secrets	<code>DATABASE_URL</code> , <code>SECRET_KEY</code> , <code>HARDWARE_API_KEY</code> (fail-fast if missing)
ASGI server	Uvicorn [standard] (uvloop + httptools + websockets)
Multi-worker	Not configured — single Uvicorn process
Database	Supabase managed Postgres (AWS EU-Central-1), asyncpg driver
Migrations	Alembic with async engine, <code>NullPool</code> , <code>compare_type=True</code>
Containerization	None — no Dockerfile or docker-compose
CI/CD	None — no pipeline configured
Redis	Local <code>redis://localhost:6379</code> — app degrades gracefully without it
Frontend	Vite dev server, separate process, CORS-allowed origins in <code>Settings</code>

Generated: 2026-04-17